Programming Project I

Scientific Conference Organization Tool

Spring 2026

Due Date: March 30, 2026, at 23:59 (PT time)

1. Objectives

This first programming assignment aims at exposing you to realistic implementation of algorithmic solutions presented in class and in particular to the **greedy algorithmic approach** in the context of Maximum Flow. Key objectives include:

- Developing teamwork skills by working in groups of **2-3 students (3 preferred)**, fostering interpersonal and project management skills.
- Conceive and implement a organizational tool, in this particular case for a specific aspects of the organization of a scientific conference and use the algorithms studied in class.
- Preparing and delivering a **concise demo presentation** to showcase the functionalities and interfaces developed, while enhancing your ability to prioritize and communicate essential information effectively. This will require you to be **succinct**, **time-conscious**, and focused on what is essential (and what is not).

This document describes the motivation of this project, the expected interface followed by a description of the problem statement and description of the demo you are expected to present. The problem statement includes a description of each task (alongside the corresponding grading). Lastly, we provide specific turn-in instructions you need to follow. **Recall, the deadline is March 30, 2026 at 23:59.**

2. Problem Motivation

In this project, you will create a tool to help the organization of a scientific committee to assign the reviews of article submissions, also known as manuscripts or paper submissions, to a set of reviewers. Reviewers assess the quality of each submission they review and either support its inclusion in the final program or propose a rejection of a submission. Given the diversity of topics of scientific conferences, it is common for reviewers to have selected areas of expertise and thus be more suitable to review submissions in topics related to their domains of expertise.

In this context, you are asked to develop a tool to help the conference organization to assign submissions to reviewer. Each submission, with its own unique identifiers (the paper ID) is on a selected topic, numbered from `1` to `K` and each reviewer, also with its own unique identification (the reviewer ID) is an expert on one of the topics (a generalization is also possible as explore later). To ensure that each paper is adequately reviewed, a submission should be review by <u>at least</u> a selected number of reviewers, each of which is only willing to review <u>up to</u> a selected number of submissions each. Note that each submission must be reviewed only by a reviewer with a specific expertise.

To help the conference organizers you are asked to develop a tool to assign (*i.e.*, decide) which reviewers will review each submission (or equivalently, which submissions will be reviewed by which reviewers).

To accomplish this task, you should formulate this problem as a network flow problem to determine one of more possible review assignments. In addition, your tool should be able to carry out a risk analysis in which the absence of one or more reviewers can jeopardize the quality of the review process. In this context,

one can manipulate (*i.e.*, increase) key problem parameters such the number of reviews per submissions (`MineviewsPerSubmission`) or the maximum number of reviews per reviewer (`MaxReviewsPerReviewer`) so that if one or more reviewers fail to meet their review goals, the number of reviews per submission is still within acceptable bounds.

3. Problem Description and Interface

To make this problem more realistic, you are provided with datasets for different instance of the problem in a file format (comma separated values) and structure in sections. The first section lists the scientific paper/article submissions with their identifier, title, author(s) and e-mail contact along with a primary topic identifier and a possible secondary topic identifier. A second list will include the reviewers, now with the identifier, name, e-mail and a primary topic identifier and a possible secondary topic identifier. To facilitate the processing additional **#** characters are used to bound the data sections in the input file. A special section, used in version of the tool's operation is also included, which will help the user control the output. Figure 1 below depicts a sample input file for a very small number of submissions and reviewers, parameters and control section described in detail below. Fields are comma separated and all characters after the special **#** character until the end of the corresponding line are to be ignored. The name of the input file is free but should have extension ".csv".

```
#Submissions
#Id, Title, Authors,E-mail,Primary,Secondary
31, "The Eternal Wheel of Reincarnation", Ralph Kholer, kholer@gmail.com, 3, 4
87, "GoDiva: A PIM Architecture", Jeff Draper, draper@usc.edu, 1,
#
#Reviewers
#Id, Name, E-mail,Primary,Secondary
1, Jaqueline N. Chame, jchame@yahoo.com, 3,
2, Mary W. Hall, mhall@hotmail.edu,      1, 4
#Parameters
MinReviewsPerSubmission, 5
MaxReviewsPerReviewer, 4
PrimaryReviewerExpertise, 1
SecondaryReviewerExpertise, 0
PrimarySubmissionDomain, 1
SecondarySubmissionDomain, 1
#Control
GenerateAssignments, 1
RiskAnalysis, 0
OutputFileName, "assign.csv"
```

Figure 1. Small problem example instance.

The first two sections of the input file description are self-explanatory, and it is assumed that there are no repeated identifiers either for the submissions or for the reviewers. The last two fields indicate the primary and possibly secondary submission domains or domains of expertise respectively of the submissions or the reviewer, being mandatory the specification of the primary fields. Files with inconsistent values of identifier or other parameters should be rejected with a summary error message indicating the issue(s) encountered.

The Parameters and Control sections include several mandatory problem control parameters, namely:

- **`GenerateAssignments`**: This numeric parameter (integer) controls the generation of an assignment of the submissions to reviewers with the following values and corresponding description:
  a) 0: The assignment is still carried out but not reported. Values other than 0 will lead to the reporting of the assignment (see the **`OutputFileName`** parameter description).
  b) 1: The assignment will exclusively consider the Primary Submission domains of the submissions and the Primary Expertise domains of the experts.

c) 2: The assignment will consider the Primary and, whenever defined, the Secondary Submission domains but exclusively the Primary Expertise domain of the reviewers.
d) 3: All Primary and whenever defined, Secondary domains of either the submissions or the reviewers can be considered to meet the minimum number of reviews per submission and the maximum number of reviews per reviewer.

- **`RiskAnalysis`**: This numeric parameter (integer) determines the type of risk analysis to be carried out with the following values and corresponding description:
  a) 0: No risk analysis is carried out.
  b) 1: Determines if the assignment is still possible (meaning if all submission review analysis requirements in terms of the number of reviews) if any one of the reviewers doesn't do all of its assignments, *i.e.*, you cannot count on these reviews at all. If an assignment, or more than one is possible, it should be reported as such. If there is an assignment that is not possible for any of the specific reviewers, the reporting must be indicate for which reviewers this is the case.
  c) K (K > 1): Determines if the assignment is still possible (meaning if all submission review analysis requirements in terms of the number of reviews) if any K of the reviewers doesn't do any of its assignments, *i.e.*, you cannot count on his reviews at all.
- **`OutputFileName`**: Indicates the non-default value of the output file name as the default (if not explicitly stated and required as the parameter **`GenerateAssignments`** is set to a non-null value) it is set to "output.csv".

The output should follow a simple human-readable format. For the assignment section of the output, each assignment line must include, the submission identifier, the corresponding reviewer followed by the matched domains, first the submission domain followed by the reviewer's domain. The dual information, which lists the set of individual submissions each reviewer is responsible for should be also shown. **All listing must be sorted**. At the end of the review assignment a "comment" line includes the total number of review assignments with the syntax "**`#Total: N`**". A sample illustration of the sample input in Figure 1 is shown in Figure 2 below.

```
#SubmissionId,ReviewerId,Match
31, 1, 3
87, 2, 1
#ReviewerId,SubmissionId,Match
1, 31, 3
2, 87, 1
#Total: 2
```

Figure 2. Small problem example assignment.

In case of an unsuccessful assignment, you need to report which submission cannot be reviewed with the minimum required reviews and which domain expertise is still required and in what quantity so that the adequate number of additional reviewers can be enlisted. The output text format should be as shown in Figure 3 below where we have assumed a minimum of 3 reviews were required.

```
#SubmissionId,Domain,MissingReviews
31, 3, 2
87, 1, 2
```

Figure 3. Unsuccessful review assignment reporting.

The section of the output regarding the listing of the "risky" reviewers, should have the syntax shown below where **`M`** indicates the risk analysis parameter value used and the list of offending reviewers should also be sorted for convenience of subsequent processing.

```
#Risk Analysis: M
1, 2
```

Figure 4. Risk Analysis Output.

The order of these sections in the output file is immaterial.

4. Problem Statement and Organization of Work

To facilitate your work, we have structured the functionalities that you are expected to develop as follows.

4.1. Basic Application Definition and Set-Up

[T1.1: 1.0 point] Develop the Assignment Tool. The first task will be to create a simple command-line menu that exposes all implemented functionalities in a user-friendly manner. This menu will also serve as a key component for showcasing the work you have developed in a short demo at the end of the project. When displaying the results, show all relevant details, including the full sequence path points from origin to destination. The program should handle exceptional cases/errors properly.

IMPORTANT: You should also provide a simple batch or command line interface where your program can be executed through a script. In this case the order of the arguments to your executable file should be as: `myProg -b input.csv output.csv` where the "`-b`" indicates an execution in batch mode and the last parameter is the output filename where all output, even the one resulting from the risk analysis is to be included. Error messages are to be directed to the console or in Unix the standard error file.

[T1.2: 1.0 point] Read and Parse the Input Data. You will need to develop basic functionality (accessible through your menu) to read and parse the provided data set files. This functionality will enable you (and the eventual user) to request information such as the list of submissions, list of reviewers and current parameter settings.

With the extracted information, you should create one (or more) appropriate data structures upon which you will carry out the requested tasks. **This data structure must be based on the data structure described in the TP lectures for graphs, for which it may make a small number of modest additions.** You must use the provided graph structure as the primary representation your solution, and you may use other auxiliary structures as needed to facilitate your implementation. Any alterations made to the original graph should be clearly justified in your presentation, ensuring that your modifications are sensible and enhance the application of the required algorithms.

[T1.3: 2.0 points] Documentation and Time Complexity Analysis. Include documentation for all implemented code using Doxygen. This should indicate the time complexity for each of the most relevant implemented algorithms.

4.2. Reviews Assignment on Primary Domains and Expertise

In this variant of the implementation, you are only required to develop a solution focusing on Primary Domains of each submission and Primary Expertise of the Reviewers only. In subsequent implementations you are to consider the other 4 variants. Still, in this context your solution should also handle basic Risk Analysis parameters.

[T2.1: 4.0 points] Basic Formulation without Any Risk Analysis
In this section you are expected to formulate the Review Assignment Problem as a Max-Flow problem. For this case, you are to assume that only primary domains of the submissions and of the reviewers are to be considered. Also, you can assume all reviewers will carry out their reviews.

The output, if any, of this assignment should be a file with the format described above.

[T2.2: 3.0 points] Basic Formulation with Risk Analysis = 1
Based on your formulation and solution to the base problem, you should explore if there is a single reviewer whose failure to do all the assignments, would hamper an assignment at all.

Describe your approach to this variant of the problem and discuss its complexity. Also, provide a non-trivial input example for which if a single reviewer does not do its reviews, the assignment of reviews to the submissions is not possible.

[T2.3: 3.0 points] Basic Formulation with Risk Analysis = K
Outline an approach (you do not need to implement it) for this example (with K > 1) using the Max-Flow algorithm as a basis or combining it with another approaches. Discuss its complexity and propose a non-trivial input example and the corresponding output that highlights the correctness and complexity of your approach.

[T2.4: 3.0 points] General Formulation without Risk Analysis
This general case explores Primary and Secondary submission domains as well as Primary and Secondary domains of expertise from the reviewers. It is a more general problem with high computational complexity if the optimal assignment is to be sought. As such, you are asked to outline an approach to solve it again using the Max-Flow algorithm or combining it with other approaches. Discuss its complexity.

5. Demo & Presentation

[T3.1: 3.0 points] Giving a presentation that is "short-and-to-the point" is increasingly important. As such, you are required to structure a short 10-minute demo of your work, highlighting key aspects of your implementation, specifically:

- Present your solution for the provided input datasets demonstrating the results requested in this project;
- Highlight your graph class, explaining the conceptual decisions that were made, including alterations to the original structure provided;
- Highlight the most important and challenging aspects of your implementations.

You should also elaborate a PowerPoint presentation to support your presentation. Instructions for preparing the PowerPoint presentation are available on Moodle.

6. Turn-In Instructions & Deadline

Submit a zip file named **DA2026_PRJ1_T<TN>_G<GN>.zip** on Moodle, where TN stand for your TP number or "Turma" (as in T01, T02, etc.) and **GN** stands for your group number, with the following content:

- **Code** folder (contains program source code)
- Documentation folder (contains html documentation, generated using **Doxygen**)
- Presentation file (**PDF format**) that will serve as a basis for the demonstration.

Late submissions, up to 24 hours and 48 hours, will incur a penalty of 10% and 25% of the grade, respectively. No submissions will be accepted 48 hours after the deadline. Exceptions apply for justified and documented technical submissions issues.

Recall, the deadline is March 30, 2026, at 23:59 (PT time).